

MLFQ Report

Multi-Level Feedback Queue Scheduling in xv6

Student	Shreshth Mehrotra
Roll No.	2025101146

1. Implementation Summary

1.1 Makefile / SCHEDULER macro

The Makefile was modified to support compile-time selection of the scheduling policy. When `SCHEDULER=MLFQ` is supplied, the kernel is compiled with `SCHED_MLFQ` and the MLFQ code path is enabled. When no scheduler flag is supplied, the original xv6 round-robin path is retained. The MLFQ build was tested using `make clean; make qemu SCHEDULER=MLFQ`, while the default build remains the RR configuration.

1.2 struct proc changes

The process structure was extended with MLFQ bookkeeping fields including the current queue, ticks consumed in the current time slice, and timing information used for evaluation. A next pointer was also added so processes can be linked inside the four scheduler queues. Timing fields store arrival/start time, first CPU time, completion time, and accumulated CPU time.

1.3 allocproc() and process creation

New processes are initialized in queue 0 with zero ticks consumed in the current slice. The same queue-0 placement is applied when the initial user process and forked children become runnable. This ensures that every newly created process starts at the highest priority as required.

1.4 Queue selection and strict-priority scheduling

Four FIFO queues are maintained for priorities 0 through 3, where queue 0 has the highest priority. The scheduler checks the queues in increasing queue-number order and removes the process at the head of the first non-empty queue. Thus, a lower-priority process cannot continue running after a higher-priority runnable process is detected at a timer-tick boundary.

1.5 Time-slice handling

The configured time slices are 1, 4, 8, and 16 ticks for queues 0, 1, 2, and 3 respectively. On every timer interrupt, the running process increments both its current-slice tick count and CPU-time count. When its slice is exhausted, it is moved to the tail of the next lower queue. A process already in queue 3 is placed back at the tail of queue 3, giving the lowest queue round-robin behavior.

1.6 Voluntary yield

The MLFQ yield path preserves the process's current queue when the process gives up the CPU before its time slice expires. The process is marked runnable and appended to the tail of the same queue, so voluntary yielding does not reduce its priority. This is separate from quantum exhaustion, which causes demotion.

1.7 Priority boosting

A priority boost is performed every 48 timer ticks. Processes found in lower-priority queues are moved to queue 0 and their current-slice tick counters are reset. Active processes that are running or sleeping are also reset to queue 0 when the boost occurs. The boost prevents long-running processes from remaining indefinitely in low-priority queues.

1.8 procdump() and debugging support

`procdump()` was extended to display the PID, process name, state, current queue, current-slice ticks, and timing bookkeeping. Additional kernel logging was used during testing to print queue transitions, quantum exhaustion, voluntary yields, and priority-boost events. These messages were used to verify the scheduler's queue movement and

boost behavior.

2. MLFQ Analysis

A custom schedulertest program was written to create four child processes with different CPU-burst workloads. The processes were classified as short, medium, long, and very-long/CPU-bound workloads, with different burst sizes and numbers of rounds. The test was run with a single CPU to make the scheduling timeline deterministic. The observed queue transitions and priority boosts were recorded from the kernel's MLFQ logging.

Process	Burst size	Rounds	Workload type
PID 4	30,000,000	20	Short CPU bursts
PID 5	60,000,000	15	Medium CPU bursts
PID 6	100,000,000	12	Long CPU bursts
PID 7	150,000,000	10	Very long / CPU-bound

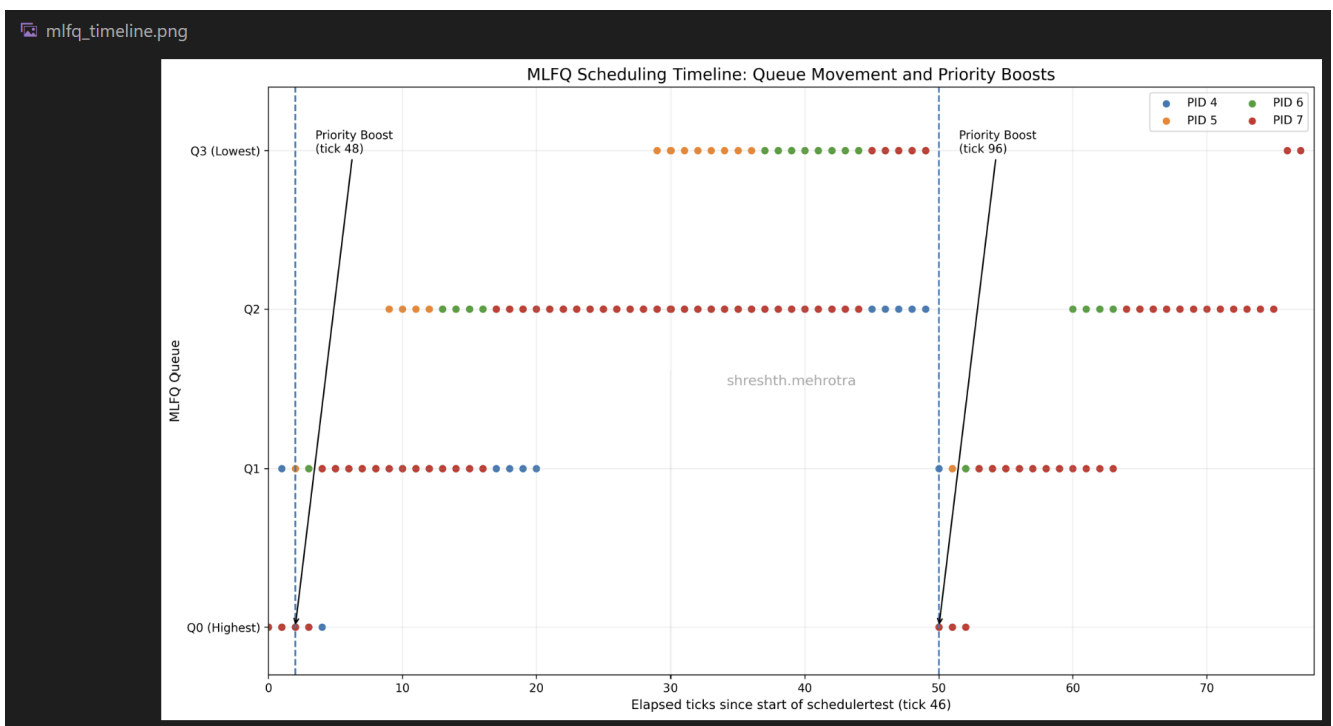


Figure 1. MLFQ queue movement observed in the test. The x-axis uses elapsed ticks relative to the workload start (absolute tick 46). Priority boosts occurred at absolute ticks 48 and 96, corresponding to elapsed ticks 2 and 50.

Interpretation

Initially, all four test processes enter the highest-priority queue Q0. CPU-bound processes gradually move from Q0 to Q1, Q2, and Q3 as they repeatedly exhaust their allotted time slices. At ticks 48 and 96, the priority-boost mechanism moves active processes back to Q0, after which CPU-bound processes again move toward lower-priority queues. This demonstrates how MLFQ favors higher-priority processes while periodically preventing long-running processes from remaining at low priority indefinitely.

Note: The current schedulertest primarily exercises CPU-burst/quantum behavior; the voluntary-yield path is implemented in the kernel but is not separately isolated by a dedicated user-space sleep/yield syscall test.

3. Cross-Scheduler Comparison

FIFO, RR, and MLFQ were evaluated using the same four-process benchmark workload. The benchmark records process arrival/start time, first CPU time, completion time, and CPU time, from which response, turnaround, and waiting metrics were calculated. The experiments were run with one CPU.

Scheduler	Avg Response (ticks)	Avg Turnaround (ticks)	Avg Waiting (ticks)
FIFO	22	41	22
RR	1.5	7.25	5
MLFQ	1.5	6.25	4

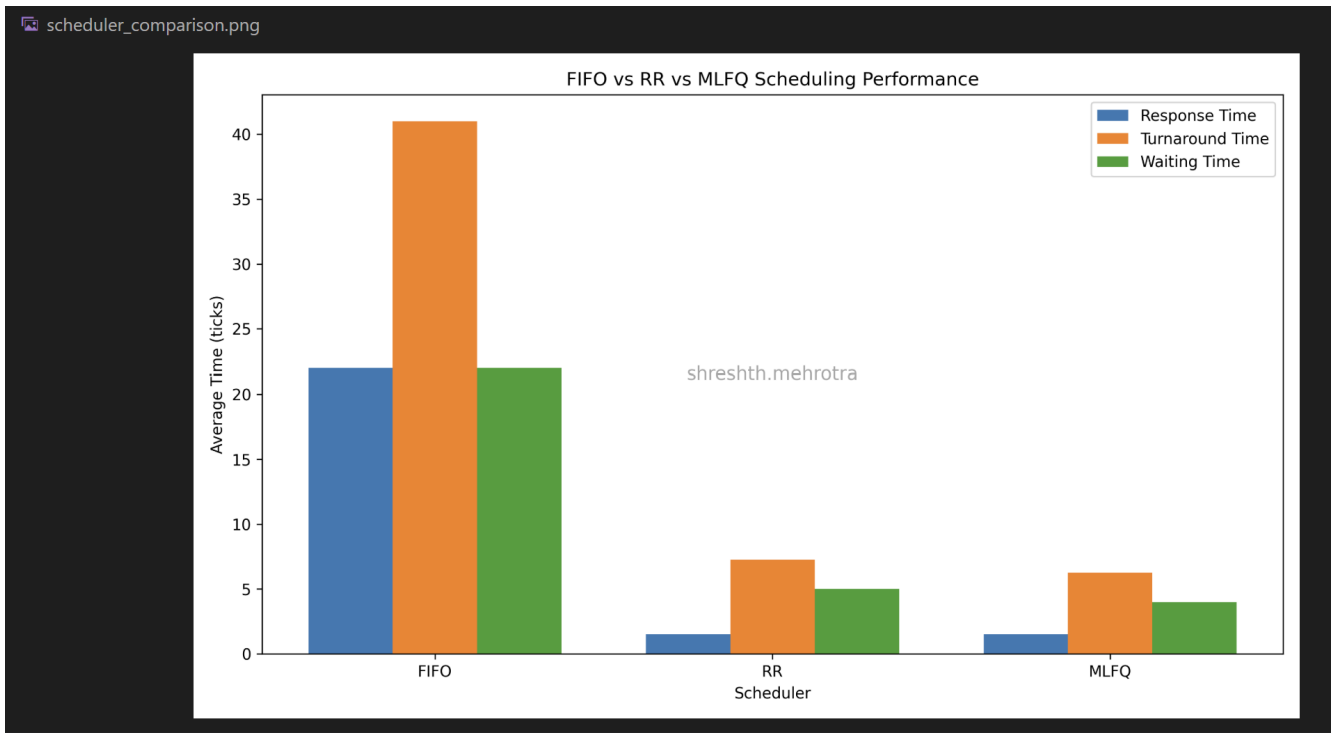


Figure 2. Average response, turnaround, and waiting times measured for the same benchmark workload under FIFO, RR, and MLFQ.

Trade-off discussion

FIFO provides simple scheduling with very low scheduling overhead, but its non-preemptive behavior causes later processes to experience much higher response and waiting times. In this experiment, FIFO had an average response time of 22 ticks, compared with 1.5 ticks for both RR and MLFQ. Round Robin provides better responsiveness because CPU time is shared among runnable processes through periodic preemption. MLFQ achieved the same average response time as RR while obtaining lower average turnaround and waiting times for this workload. This is consistent with MLFQ allowing CPU-bound processes to gradually move to lower-priority queues while maintaining higher priority for processes that need more immediate CPU access. The periodic priority boost also prevents processes from remaining indefinitely in low-priority queues. Overall, FIFO is simple but less responsive, RR provides fair time sharing, and MLFQ offers a balance between responsiveness, priority adaptation, and CPU-bound workload performance.

4. Build and Execution Commands

Default (Round Robin): make clean; make qemu

MLFQ: make clean; make qemu SCHEDULER=MLFQ

The first command uses the default xv6 round-robin scheduler. The second explicitly selects the MLFQ implementation through the compile-time scheduler macro.

5. Conclusion

The xv6 kernel was extended with a compile-time selectable MLFQ scheduler while retaining the original RR path when no scheduler flag is supplied. The implementation uses four priority queues, queue-specific time slices, strict-priority selection, quantum-based demotion, same-queue voluntary yielding, queue-3 round robin, and periodic 48-tick priority boosts. Kernel instrumentation and the custom scheduler test were used to verify queue transitions and boosts. The cross-scheduler benchmark shows that MLFQ achieved lower turnaround and waiting times than RR for the tested workload while matching RR's response time, whereas FIFO had substantially larger response and waiting times.